CS459 Assignment 02 - Yash Barve

Name: Yash Harshal Barve
Email: yhbarve@uwaterloo.ca
Student ID: 20930353

Question 01

(a)

- All the four packets are permitted, as described in the following table.

Packet	Direction	Src Addr	Dest Addr	Protocol	Dest Port	Action	Rule
1	In	192.168.3.4	172.16.1.1	TCP	443	Permit	A
2	Out	172.16.1.1	192.168.3.4	TCP	1234	Permit	B
3	Out	172.16.1.1	192.168.3.4	TCP	443	Permit	C
4	In	192.168.3.4	172.16.1.1	TCP	1357	Permit	D

(b) i

- Both the packets succeed.

Packet	Direction	Src Addr	Dest Addr	Protocol	Dest Port	Action	Rule
5	In	10.1.2.3	172.16.3.4	TCP	8080	Permit	D
6	Out	172.16.3.4	10.1.2.3	TCP	5150	Permit	B

- However, note that packet 5 is sent as part of initiating the connection with the server. Ideally, it should be sent to port 443 (standard port for HTTPS). Here, it is sent to port 8080. But because it matches Rule D, the packet will be permitted.

(b) ii

- Yes, the attack will succeed. This is because both packets 5 & 6 are permitted, allowing the external attacker to establish a connection to the server on port 8080.

(c)

- Packets 1, 2, 3, 4 are permitted while 5 and 6 are denied.

Packet	Direction	Src Addr	Dest Addr	Protocol	Src Port	Dest Port	Action	Rule
1	In	192.168.3.4	172.16.1.1	TCP	1234	443	Permit	A
2	Out	172.16.1.1	192.168.3.4	TCP	443	1234	Permit	B
3	Out	172.16.1.1	192.168.3.4	TCP	1357	443	Permit	C
4	In	192.168.3.4	172.16.1.1	TCP	443	1357	Permit	D
5	In	10.1.2.3	172.16.3.4	TCP	5150	8080	Deny	E
6	Out	172.16.3.4	10.1.2.3	TCP	8080	5150	Deny	E

Question 02

(a)

- The application server verifies the client by decrypting the service ticket using its own secret share key to extract the session key.
- It then uses the session key to decrypt the client's authenticator, which contains a timestamp and the client ID.
- If the client ID matches and the timestamp is fresh, authenticity is proven.

(b)

- The authentication server encrypts TGT using the secret key K_{TGS} of the ticket granting server.
- The client or any outside adversary does not have access to K_{TGS}.
- Therefore, any alteration to the TGT will corrupt the data, causing decryption to fail at the TGS stage.

(c)

- The Kerberos protocol uses an authenticator containing the current timestamp.
- The server checks this timestamp against its local clock, and if the message window is outside an acceptable range, the packet is dropped
- Additionally, servers maintain a replay cache within that time range to catch duplicate messages.

(d)

1. Client Secret Key: Known only to the client and the authentication server (AS). It is used to decrypt the initial response packet from A, which contains the ticket granting server (TGS) session key and the TGT.
2. TGS Secret Key: Known only to the AS and the TGS. It is used by the AS to encrypt the TGT and by the TGS to decrypt it.
3. Application Server Secret Key: Known only to the TGS and the specific application server. It is used by the TGS to encrypt the final service ticket (ST).
4. TGS Session Key: A temporary session key generated by the KDC for secure communication between the client and the TGS. It encrypts the TGS request details and the authenticator.
5. Service Session Key: A temporary session key generated by the TGS for secure application traffic between the client and the application server.

(e)

- The authentication server distributes the session key as two copies:
  1. The first copy is intended for the TGS. It is wrapped inside the TGT and encrypted under the TGS secret key.
  2. The second copy is intended for the client. It is placed outside the TGT but encrypted under the client's secret key. The client can decrypt it and read the session key.

Question 03

(a)

- YES.

- SSL/TLS prevents replay attacks using the random nonce values generated by the client and server during the handshake phase.
- The client and server also have a shared key that is established during the asymmetric key exchange phase.
- These three keys are used to derive the final session keys.
- If an adversary intercepts a handshake session and replays the messages later, the new session will use a fresh server nonce.
- This would cause the final session key to mismatch, forcing the target to terminate the connection.

(b)

- YES.
- Man-in-the-middle attacks are blocked during the SSL/TLS handshake via asymmetric authentication using PKI and CAs.
- The server transmits its digital server certificate and signature directly to the browser. The client passes the certificate chain and verifies that the host name matches the requested URL.
- The server must also sign all handshake messages using its private key corresponding to the verification key in the certificate.
- An adversary cannot replicate the signature without having the server's private key. This prevents any man-in-the-middle attacks.

(c)

- YES.
- Once the handshake is successfully completed, all the data sent over the connection is protected.
- SSL/TLS record protocol takes raw application layer data and encrypts it using strong symmetric ciphers. It also adds a MAC for integrity.
- Therefore, any data traveling across the connection is encrypted. So any adversary sniffing the network will not have access to the original data.

(d)

- YES, but not directly.
- SSL/TLS prevents IP spoofing indirectly using the MACs and sequence numbers that every data packet includes.
- An attacker does not know the secret session keys and therefore cannot generate a valid payload MAC or the exact or correct sequence numbers.
- Therefore, the receiving hosts will fail to verify the block and discard the bogus data.

(e)

- NO.
- SSL/TLS cannot mitigate a SYN flood attack.
- This is because SSL/TLS operates strictly at the transport layer above the raw TCP execution.
- In SYN flooding, the attacker sends a lot of TCP SYN packets but never completes the handshake. Because the TCP handshake is never finished, the connection is never fully established.
- TLS relies on a TCP connection to send its own handshake messages. Therefore, in this case, TLS is unable to prevent SYN flooding.

Programming Question 01

(a)

- Yes, there is explicit evidence linking Mr. Raccoon to the article.
- After filtering out the background traffic, I discovered a Python server `SimpleHTTP/0.6 Python/3.8.10` at IP address `4.227.186.176:8000`.
- A `Wget/1.20.3` machine (belonging to Mr. Raccoon) made multiple GET requests to fetch the contents of a folder named `/fishy`.
- Upon inspection, I found a file named `todo.txt`, which contains a checklist about the article.

(b)

- HTML Hex Color Encoding
    - This is the primary steganographic technique employed in this case.

    - After discovering the `/fishy` folder, I also saw GET requests made for 423 web documents named from `file0.html` to `file422.html`.
    - Each file contains exactly 10 styled elements containing hex attributes.
    - If these hex attributes are concatenated, the resulting string is a raw binary byte stream representing a complete JPEG digital image.
    - The `todo.txt` file also contains `Brush up on web colors - DONE`, which indicates the use of hex-color encoding.
- Other plausible techniques include:
    1. HTML comment encoding: Hiding bits in comments in the HTML
    2. HTTP headers: Storing data in custom headers, or cookies.

(c)

- I performed the following function:
    - Loop through each of the 423 files
    - Pull the 10 hex codes in the exact order they appear
    - Convert the hex characters into raw binary bytes
        - Eg: `ffd8ff -> ff d8 ff`
    - Append all the bytes into a single file stream
- I wrote a script to do this for me. Here's what the script does:
    - The script filters the packets for the TCP layer, Raw data payload, and 'Columbia Chronicles Draft' in the payload.
        - The payload text was identified upon inspecting a few packets manually.
    - To handle out of order arrival, I used a dictionary to order the data in the correct chronological order.
    - For each file, a regular expression extracts the ten hex attributes, and removes the leading `#` symbol.
    - The parsed hex characters are converted to binary bytes using the in-built `bytes.fromhex()` function.
    - After all the packets are read, the script concatenates the bytes into a single stream and writes the final payload to the file `recovered`.
    - After renaming this output file to `recovered.jpeg`, we can see the artifact.

Note:

- The conclusive artifact is attached as part of the submission zip file.

- The script `get-artifact` is also attached as part of the submission zip file. The script is written in Python3.
- To run the script:
  - `source .venv/bin/activate`
  - `pip install scapy`
  - `./get-artifact p-q1-capture.pcap`

Programming Question 02

New Features

	Feature	How does this feature increase accuracy?
1	`trace_duration`	Some pages finish loading quickly while others take much longer
2	`outgoing_fraction`	Captures whether a page is interaction-heavy or content-heavy.
3	`first50_incoming_cells`	How much content the server pushes back early in the load
4	`longest_incoming_cells`	Longest streak of incoming cells indicates larger content bursts, which could indicate content-heavy websites
5	`switches`	Direction switches divided by total cells. Measures how much back-and-forth the exchange is.

Best Feature Set

	Feature
1	`total_cells`
2	`trace_duration`
3	`outgoing_fraction`
4	`switches`
5	`first20_outgoing_cells`
6	`first50_incoming_cells`
7	`mid_incoming_cells`
8	`last50_outgoing_cells`

	Feature
9	`outgoing_before_first_incoming`
10	`mean_gap`
11	`gap_std`
12	`longest_outgoing_cells`
13	`longest_incoming_cells`

The highest classification accuracy achieved with $k = 5$ = 0.67 = 67%
The overall highest accuracy achieved = 0.875 = 87.5% ($k = 1$)

Influence of Neighbours

k	1	2	5	10	15
ACC	0.875	0.79	0.67	0.54	0.45

- Accuracy decreases as k increases.
- As k increases, the majority vote is affected by neighbours that are further away. These additional neighbours add more noise, diluting the correct label.
- Therefore, accuracy drops as the number of neighbours increase.

Note

- `fextractor.py` is included in the submission file.
